

AUDIT

Rapport d'audit sur les problèmes récurrents du chiffre d'affaires

Table des matières

1. Résumé Exécutif	5
1.1. Objectif de l'audit.....	5
1.2. Conclusions principales.....	5
1.3. Recommandations principales	5
1.4. Portée de l'audit.....	6
2. Introduction.....	7
2.1. Contexte.....	7
2.2. Objectifs de l'audit	7
2.3. Périmètre de l'audit	7
2.4. Méthodologie	8
2.4.1. Analyse du schéma d'architecture des données	8
2.4.2. Exploration des données	8
2.4.3. Modélisation relationnelle	8
2.4.4. Création d'un prototype SQL.....	8
2.4.5. Analyse du chiffre d'affaires	8
2.4.6. Analyse des logs	8
3. Description de l'Entité Auditée.....	9
3.1. Présentation de l'organisation	9
3.2. Processus ou systèmes audités.....	9
3.2.1. Architecture auditée.....	9
3.2.2. Description des tables auditées.....	10
4. Constats de l'audit.....	13
4.1. Constat 1 : Désynchronisation entre la base SQL Server et le cube OLAP	13
4.1.1. Description.....	13
4.1.2. Impact.....	13
4.1.3. Preuve	14
4.2. Constat 2 : Absence de supervision technique de la base de données.....	14
4.2.1. Description.....	14

4.2.2.	Impact.....	14
4.2.3.	Preuve	14
4.3.	Constat 3 : Présence de données personnelles ou sensibles dans le cube OLAP	15
4.3.1.	Description.....	15
4.3.2.	Impact.....	15
4.3.3.	Preuve	15
4.4.	Constat 4 : Documentation insuffisante du modèle de données.....	15
4.4.1.	Description.....	15
4.4.2.	Impact.....	15
4.4.3.	Preuve	16
4.5.	Constat 5 : Qualité, nommage et typage des données perfectibles.....	16
4.5.1.	Description.....	16
4.5.2.	Impact.....	16
4.5.3.	Preuve	16
4.6.	Évaluation des risques	17
5.	Analyse des Résultats.....	18
5.1.	Analyse des écarts.....	18
5.2.	Causes des écarts	18
6.	Recommandations	19
6.1.	Recommandation 1 : Mettre en place une réconciliation entre la base SQL Server et le cube OLAP	19
6.2.	Recommandation 2 : Mettre en place une supervision technique.....	22
6.3.	Recommandation 3 : Restreindre les données personnelles dans la couche BI.....	23
6.4.	Recommandation 4 : Formaliser le dictionnaire de données, documenter le modèle relationnel et les règles de calcul.....	24
6.5.	Recommandation 5 : Améliorer le typage, le nommage et la qualité des données.....	24
6.6.	Priorisation des recommandations.....	25
7.	Conclusion.....	26
8.	Annexes	27
	Annexe 1 : Glossaire	27
	Annexe 2 : Dictionnaires des données	29

Annexe 3 : Script de création des tables du POC.....	30
Annexe 4 : Requête SQL du calcul du chiffre d'affaires du 14 août.....	32
Annexe 5 : Requête SQL du calcul du chiffre d'affaires par client pour le top 10 des clients	33
Annexe 6 : Requête SQL du calcul de la part du chiffre d'affaires encaissé par employé.....	34
Annexe 7 : Requête SQL pour retrouver l'écart du chiffre d'affaires	36
Annexe 8 : Procédure de génération aléatoire de ventes.....	37
Annexe 9 : Requête SQL pour la création des tables logs.....	38
Annexe 10 : Requête SQL fonction trigger logs_employe.....	41
Annexe 11: Requête SQL fonction trigger logs_ventes.....	43
Annexe 12: Requête SQL fonction trigger logs_client.....	45
Annexe 13: Requête SQL fonction trigger logs_produits.....	47
Annexe 14 : Légende matrice évaluation des risques	50
Annexe 15 : Sources	51

Table des figures

Figure 1: Architecture data de SuperSmartMarket	9
Figure 2: Schéma relationnel : Modélisation en étoile des données.....	10

1. Résumé Exécutif

1.1. Objectif de l'audit

L'audit a été mené à la demande du responsable Business Intelligence de SuperSmartMarket afin d'identifier l'origine des incohérences observées dans les chiffres d'affaires affichés dans Power BI.

L'exemple déclencheur concerne le chiffre d'affaires du 14 août 2024, qui initialement était à 275 186,59 €, puis réévalué à 284 243,88 € deux jours plus tard, alors que les magasins étaient fermés le 15 août 2024.

L'objectif principal est donc de comprendre pourquoi l'historique du chiffre d'affaires évoluait alors qu'il devrait normalement rester stable une fois la journée terminée.

1.2. Conclusions principales

L'audit a permis de mettre en évidence les points suivants :

- Le fichier extrait du cube OLAP contient les données nécessaires à la reconstruction d'un modèle relationnel de type étoile,
- Le chiffre d'affaires correct du 14 août 2024 est confirmé à 284 243,88€,
- L'écart constaté entre les deux montants est de $284\,243,88 - 275\,186,59 = 9\,057,29\text{€}$
- L'analyse des logs montre que certaines ventes rattachées à la date commerciale du 14 août 2024 ont été insérées en base de données le 15 août 2024. Cela suggère un retard d'intégration entre la base source et le cube OLAP. La variation du chiffre d'affaires est donc probablement liée à un problème de délai d'intégration ou de synchronisation des données,
- Des données sensibles ou inutiles à l'analyse BI figurent dans les tables, notamment les hash_mdp, les noms, prénoms et adresses e-mail des employés.

1.3. Recommandations principales

Les recommandations principales sont les suivantes :

1. Mettre en place une réconciliation entre la source SQL Server et le cube OLAP afin de détecter automatiquement les écarts de chiffre d'affaires,
2. Mettre en place une supervision technique de la base de données,
3. Renforcer la gouvernance des données sensibles en supprimant les informations personnelles non nécessaires pour l'analyse,
4. Documenter le modèle et le dictionnaire de données,
5. Améliorer le nommage, le typage et la qualité des données.

1.4. Portée de l'audit

L'audit couvre :

- La compréhension de l'architecture data de SuperSmartMarket (SQL Server →Cube OLAP→Power BI),
- L'analyse du fichier extrait du cube OLAP du 14 août 2024 (fichier Excel qui contient 5 tables : ventes, produits, clients, calendrier, employés,
- La création d'un dictionnaire des données à partir de l'analyse de ce fichier Excel,
- La modélisation des tables sous forme d'un schéma relationnel en étoile,
- La création d'un prototype de base de données PostgreSQL à partir des données du fichier Excel,
- L'étude des logs pour identifier l'origine des écarts.

2. Introduction

2.1. Contexte

SuperSmartMarket est une chaîne de supermarchés en plein essor en France. L'entreprise souhaite se démarquer en mettant l'accent sur l'hyper-personnalisation de l'expérience en magasin avec des coupons promotionnels personnalisés, des recommandations basées sur les historiques d'achat, des recettes personnalisées en fonction des achats, etc.

Pour atteindre cet objectif, l'entreprise s'appuie sur les technologies de pointe qu'elle intègre dans ses supermarchés et notamment sur un environnement de données composé :

- D'une base de données opérationnelle SQL Server,
- D'un entrepôt de données de type OLAP via Azure Analysis Services,
- D'un outil de restitution Power BI,
- D'un site internet et d'un ERP.

Cependant le pôle de Business Intelligence rencontre un problème majeur : les indicateurs historiques de chiffre d'affaires ne sont pas stables. Un chiffre d'affaires calculé un jour peut varier lorsqu'il est consulté plus tard.

Cette instabilité compromet la confiance dans les tableaux de bord et limite la capacité de l'entreprise à piloter son activité de manière fiable.

2.2. Objectifs de l'audit

Les objectifs spécifiques de l'audit sont les suivants :

- comprendre l'architecture générale des flux de données,
- analyser la structure des fichiers issus du cube OLAP,
- produire un dictionnaire des données,
- proposer un schéma relationnel à partir du fichier plat,
- créer une base de données prototype en local,
- vérifier le chiffre d'affaires du 14 août 2024,
- calculer le chiffre d'affaires par client pour les 10 meilleurs clients,
- calculer la part de chiffre d'affaires encaissée par employé,
- analyser les logs fournis afin d'expliquer les écarts observés,
- formuler des recommandations d'amélioration.

2.3. Périmètre de l'audit

Le périmètre de l'audit porte sur :

- Tables issues de l'extraction OLAP (vente, produit, client, calendrier, employé)

- Les données de logs sur la base de données pour la journée du 14 août 2024 (actions réalisées, dates des actions, types d'actions, champs modifiés etc)
- Les indicateurs à vérifier (chiffre d'affaires global, par client, par employé, écart)

Les traitements ETL ne font pas partie du périmètre de l'étude, donc il n'y a pas d'informations sur l'alimentation du cube OLAP depuis SQL Server.

2.4. Méthodologie

2.4.1. Analyse du schéma d'architecture des données

Le schéma d'architecture des données fourni par l'entreprise est analysé afin de comprendre les processus de création de données.

2.4.2. Exploration des données

Le fichier Excel correspondant à l'extraction des données du cube OLAP est analysé notamment à l'aide de la bibliothèque pandas de Python.

Les tables analysées contiennent des informations sur les ventes, les produits, les clients, le calendrier, les employés.

2.4.3. Modélisation relationnelle

Un modèle relationnel a été construit à l'aide de SQL Power Architect à partir de données à plat, avec une table de faits centrale et plusieurs tables de dimensions.

La table centrale est la table de vente, reliée aux dimensions client, produit, employé et calendrier.

2.4.4. Création d'un prototype SQL

Une base de données PostgreSQL locale a été créée afin de reproduire les analyses en SQL.

Les tables ont été créées à l'aide des scripts SQL générés par SQL Power Architect, puis alimentées avec les données extraites.

Les scripts PostgreSQL présentés en annexe servent à reproduire localement le comportement observé et à tester les contrôles. Les mécanismes devront être adaptés à SQL Server, Azure Analysis Services et Power BI.

2.4.5. Analyse du chiffre d'affaires

En utilisant des requêtes SQL, les chiffres issus du prototype SQL ont été comparés avec les montants observés dans Power BI.

2.4.6. Analyse des logs

Les logs ont été importés et analysés afin d'identifier les opérations réalisées sur les tables, notamment les insertions tardives de ventes. Le comportement de remplissage automatique des logs a été simulé avec des fonctions trigger (voir les requêtes en annexe). Ce ne sont pas des logs techniques.

3. Description de l'Entité Auditée

3.1. Présentation de l'organisation

SuperSmartMarket est une entreprise de grande distribution opérant en France. Son modèle repose sur une modernisation de l'expérience d'achat, avec un usage accru des données pour mieux comprendre et personnaliser la relation client. Le pôle Data Support garantit la qualité et la sécurité des données pour l'ensemble de l'entreprise. Elle utilise une planification de processus avec des solutions Microsoft, un entrepôt de données OLAP et le support de l'application Microsoft Power BI.

3.2. Processus ou systèmes audités

3.2.1. Architecture auditée

L'audit concerne le processus de production des données du pôle Business Intelligence de l'entreprise.



Figure 1: Architecture data de SuperSmartMarket

L'architecture montre 3 étapes de circulation des données dans le système informatique de l'entreprise:

1. La source correspondant à la base de données en live (SQL Server). Les données sont générées et stockées en temps réel. Ex: transactions, ventes, clients, etc. C'est le cœur du système.

2. À partir de cette source, on part :

Vers le Cube OLAP (Azure Analysis Services) → les données sont extraites et organisées pour l'analyse. Le cube permet de les croiser selon plusieurs dimensions (temps, produit, rayon...) afin de produire des rapports et des statistiques.

Vers le site internet et l'ERP → les mêmes données alimentent les outils opérationnels du quotidien (le site web de l'entreprise et son logiciel de gestion interne).

3. La restitution avec le serveur Dataviz (Power BI): Le cube OLAP alimente enfin Power BI, qui permet aux utilisateurs de visualiser les données sous forme de tableaux de bord et de graphiques interactifs pour aider à la prise de décision.

3.2.2. Description des tables auditées

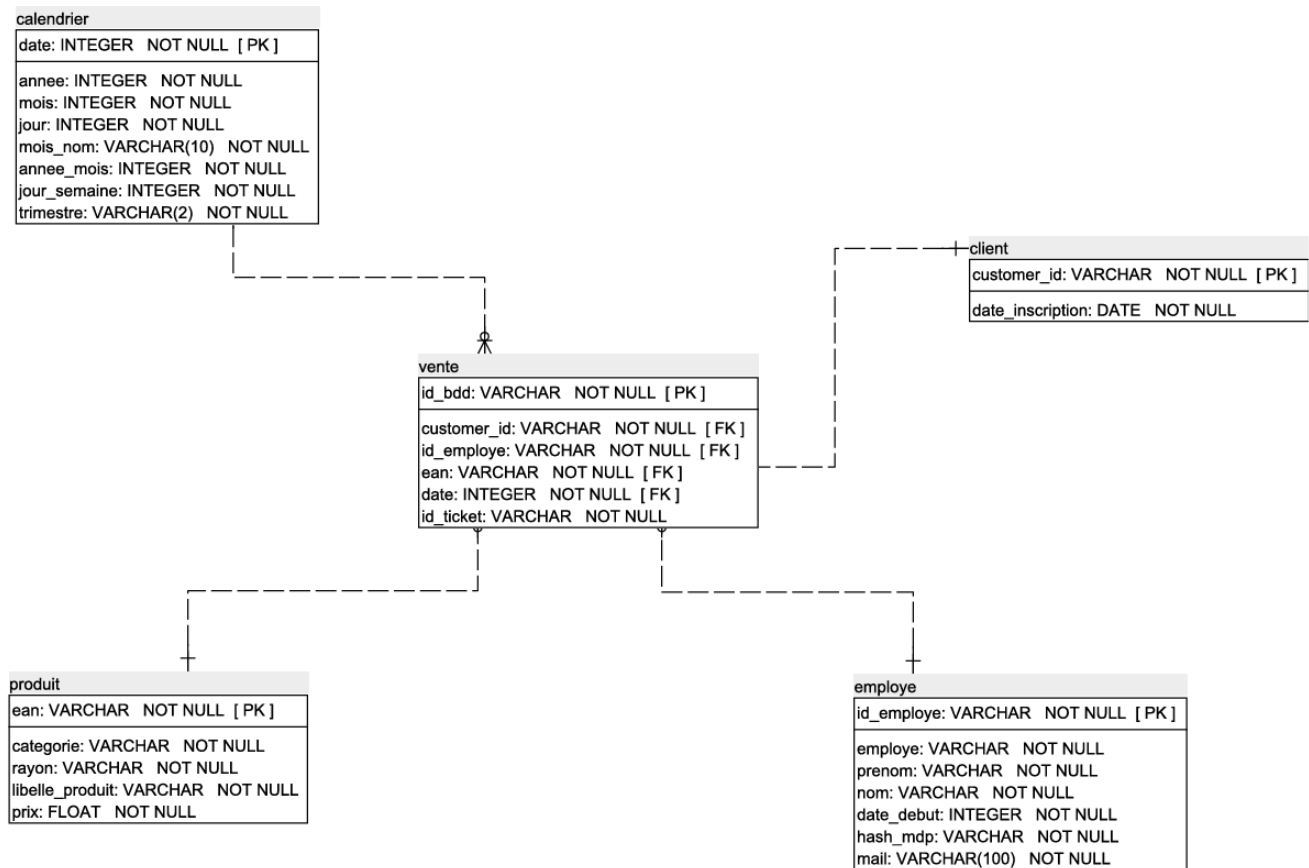


Figure 2: Schéma relationnel : Modélisation en étoile des données

Le modèle en étoile consiste à relier plusieurs tables de dimensions à une table de faits centrale. Son principal avantage est la flexibilité : plutôt que de modifier une table de faits existante, on enrichit le modèle en ajoutant simplement de nouvelles dimensions (marketing, conseiller commercial, etc.), offrant de nouveaux axes d'analyse.

Table vente

La table vente contient les lignes de transactions d'achat.

Champ	Description
id_bdd	Identifiant unique de la ligne de vente
customer_id	Identifiant du client
id_employe	Identifiant de l'employé ou caissier
ean	Code-barres du produit vendu
date	Identifiant de la date de l'achat
id_ticket	Identifiant du ticket de caisse

Cette table est assimilable à une table de faits.

Table produit

La table produit contient les informations descriptives des produits.

Champ	Description
ean	Code-barres unique du produit
categorie	Catégorie du produit
rayon	Rayon du magasin
libelle_produit	Nom ou description du produit
prix	Prix de vente du produit

Table client

La table client contient les informations relatives aux clients identifiés.

Champ	Description
customer_id	Identifiant unique du client
date_inscription	Date d'inscription du client, probablement au programme de fidélité

Table calendrier

La table calendrier permet de décomposer les dates en attributs temporels utiles à l'analyse.

Champ	Description
date	Identifiant numérique de la date
annee	Année
mois	Mois
jour	jour
mois_nom	Nom du mois
annee_mois	Identifiant du couple année-mois
jour_semaine	Jour de la semaine
trimestre	Trimestre Q1, Q2, Q3, Q4

Table employe

La table employe contient les informations relatives aux employés.

Champ	Description
id_employe	Identifiant unique de l'employé
employe	nom abrégé
prenom	Prénom de l'employé
nom	Nom de l'employé
date_debut	Identifiant de la date de début
hash_mdp	Hash du mot de passe
mail	Adresse e-mail professionnelle

À noter que cette table contient des données sensibles ou personnelles dont la présence dans un environnement analytique doit être questionnée.

4. Constats de l'audit

4.1. Constat 1 : Désynchronisation entre la base SQL Server et le cube OLAP

4.1.1. Description

Le chiffre d'affaires du 14 août 2024 a évolué entre deux consultations Power BI :

- Première valeur observée : 275 186,59€
- valeur ultérieure observée : 284 243,88€.

L'audit confirme que le chiffre d'affaires final du 14 août est bien de 284 243,88€.

L'écart entre les deux valeurs est de 9 057,29€.

Suspicion d'une désynchronisation ou d'un retard d'intégration entre la source de données SQL Server et le cube OLAP. Lors de la première consultation le cube n'avait pas encore intégré l'ensemble des ventes présentes dans SQL Server. Power BI affichait donc un chiffre d'affaires incomplet.

Dans l'environnement Microsoft, lors du traitement d'un modèle Analysis Services, le moteur se connecte à la source de données (SQL Server), exécute les requêtes associées aux partitions, puis charge les données nouvelles ou mises à jour dans le modèle tabulaire (ici le cube OLAP). Les partitions servent à réduire le temps de rafraîchissement, économiser de la mémoire, améliorer les performances, et permettent des traitements en parallèle. Une hypothèse technique à vérifier concerne le mode de rafraîchissement du cube, notamment l'existence éventuelle de partitions. Une mauvaise configuration de ces traitements pourrait expliquer qu'une partie des ventes du 14 août n'ait été intégrée qu'ultérieurement. Cette hypothèse nécessite toutefois l'analyse des paramètres de traitement Azure Analysis Services et des historiques de refresh.

N'ayant pas accès aux processus ETL, l'origine de cette désynchronisation reste à prouver. En effet, on ne sait pas comment est alimenté le cube OLAP (par paquet de données, taille des paquets, période d'alimentation) et l'on ne connaît pas l'état du système au moment du transfert des données (requêtes échouées, coupure d'électricité, saturation mémoire, etc.).

4.1.2. Impact

Cette instabilité peut avoir plusieurs conséquences :

- perte de confiance des utilisateurs dans Power BI,
- décisions commerciales prises sur des chiffres incomplets,
- difficultés de clôture journalière,
- risque de reporting contradictoire entre équipes,
- perte de crédibilité de l'équipe BI.

4.1.3. Preuve

L'analyse des ventes et des logs montre que certaines ventes associées au 14 août 2024 ont été insérées dans la base SQL Server mais n'avaient pas encore été prises en compte dans le cube OLAP au moment de la première consultation.

L'écart de chiffre d'affaires correspond précisément à la somme des ventes insérées tardivement.

L'audit met en évidence un retard d'intégration des ventes dans la chaîne décisionnelle. En l'absence de mécanisme de réconciliation entre SQL Server et le cube OLAP, l'hypothèse la plus probable est une désynchronisation temporaire ou un rafraîchissement incomplet du cube.

4.2. Constat 2 : Absence de supervision technique de la base de données

4.2.1. Description

La détection des anomalies repose actuellement sur l'observation des incohérences métier (ex. : variation du chiffre d'affaires), et non sur une surveillance de l'état technique du système.

Les logs permettent de tracer les modifications de données dans le cube, mais ils ne suffisent pas à superviser la chaîne BI complète. Une supervision technique distincte devrait couvrir les processus ETL, les rafraîchissements du cube, les erreurs d'exécution et les durées de traitement.

4.2.2. Impact

L'absence de supervision technique a plusieurs impacts :

- Détection tardive des anomalies : Les problèmes techniques (retards de chargement, blocages, requêtes lentes) ne sont identifiés qu'après impact sur les indicateurs métiers.
- Instabilité des pipelines de données : des dysfonctionnements de la base de données peuvent passer inaperçus, entraînant des retards ou des incohérences dans les données.
- Altération de la qualité des données BI : les tableaux de bord Power BI peuvent être alimentés de données incomplètes sans alerte préalable.
- Difficulté d'investigation : en l'absence de supervision, il est plus difficile d'identifier la cause d'un incident.
- Dégradation des performances : des requêtes inefficaces peuvent ralentir la base sans être détectées.

4.2.3. Preuve

L'analyse des anomalies (notamment les insertions tardives du 15 août 2024) a été réalisée manuellement à partir des logs des tables du cube. Aucun dispositif de supervision technique n'a été

fourni dans le cadre de l'audit. Cela indique soit une absence de supervision, soit une supervision insuffisamment exploitée ou documentée.

4.3. Constat 3 : Présence de données personnelles ou sensibles dans le cube OLAP

4.3.1. Description

La table employe contient plusieurs informations sensibles ou à caractère personnel (nom, prénom, adresse e-mail, hash de mot de passe).

La présence du champ hash_mdp dans une extraction destinée à l'analyse BI n'est pas nécessaire.

Même si un hash n'est pas un mot de passe en clair, c'est une donnée sensible qui doit être protégée.

4.3.2. Impact

Les risques sont les suivants :

- non-respect du principe de minimisation des données,
- exposition inutile de données personnelles,
- risque de compromission en cas de fuite,
- non-conformité potentielle aux exigences de sécurité et de protection des données (RGPD).

4.3.3. Preuve

Présence explicite de la colonne hash_mdp dans la table employe et les logs.

4.4. Constat 4 : Documentation insuffisante du modèle de données

4.4.1. Description

Le mail initial indique que la base de données n'a pas été documentée. Le dictionnaire des données et le schéma relationnel ont dû être reconstruits à partir de l'extraction du cube OLAP.

4.4.2. Impact

L'absence de documentation peut entraîner :

- mauvaise compréhension des données,
- mauvaise affectation des clés étrangères et primaires,
- erreurs de calcul,
- difficultés d'intégration pour les nouveaux collaborateurs,
- complexité accrue lors des évolutions du système.

4.4.3. Preuve

Hugo indique dans son mail initial : « Nous n'avons malheureusement jamais documenté notre base de données. »

4.5. Constat 5 : Qualité, nommage et typage des données perfectibles

4.5.1. Description

Certaines colonnes nécessitent des conversions ou des traitements spécifiques :

- dates non stockées sous format datetime dans la table client,
- colonnes nommées « date » alors qu'il s'agit d'identifiants qui servent à faire le lien avec la table calendrier,
- nommage des colonnes non homogène : il y a un mélange de majuscules, minuscules, espace, non-espace, français, anglais,
- nommage peu explicite (ex : id_bdd)
- les libellés des produits ne sont pas normalisés (des fois des accents, d'autres fois non, l'ordre des informations n'est pas homogène)
- les colonnes « date » sont en chiffres (ex :43831). Pour une lecture plus aisée, mettre directement la date (ex : 2020-01-01) en format date aurait été moins source de confusion.
- colonne avec plusieurs formats de données en même temps dans les logs (Varchar, Integer, Date).

4.5.2. Impact

Ces problèmes peuvent générer :

- erreurs d'import,
- mauvaise interprétation des dates,
- échecs de chargement,
- erreurs dans les jointures,
- calculs incorrects,
- des manipulations supplémentaires consommatrices de temps.

4.5.3. Preuve

Dans les logs, la colonne detail contient à la fois des dates, des prix, des numéros clients, des numéros de tickets. La colonne id_ligne contient des chiffres et des caractères. Dans les logs et les tables de la base de données, la colonne nommée « date » prête à confusion car elle laisse croire que c'est la date sous format numérique alors qu'il semble plutôt s'agir d'un identifiant lié à la table calendrier.

4.6. Évaluation des risques

Il s'agit d'évaluer les risques associés à chaque constat (impact, probabilité).

La méthode utilisée est une méthode qualitative : risque = probabilité x impact. Voir annexe pour plus de détails.

ID	Vulnérabilité / Constatation	Type de vulnérabilité	Probabilité (Faible/Moyenne/Élevée)	Impact (Faible/Moyen/Élevé)	Niveau de Risque (auto)
VUL-001	Désynchronisation SQL Server / OLAP	Reporting erroné / Données incomplètes dans Power BI	Élevée	Élevé	CRITIQUE
VUL-002	Absence de supervision de la base de données	Origine non identifiable des anomalies	Élevée	Élevé	CRITIQUE
VUL-003	Données sensibles dans la BI	Risque sécurité / conformité	Moyenne	Élevé	ÉLEVÉ
VUL-004	Documentation insuffisante	Mauvaise interprétation des données	Élevée	Moyen	ÉLEVÉ
VUL-005	Qualité, nommage et typage des données	Erreurs de chargement ou de calcul	Moyenne	Moyen	MOYEN

5. Analyse des Résultats

5.1. Analyse des écarts

Bonnes pratiques attendues :

- Selon la règle de gestion attendue par le pôle BI, le chiffre d'affaires journalier devrait être stabilisé à J+1. Toute évolution ultérieure devrait être tracée, justifiée et distinguée du chiffre d'affaires initialement publié. Or, il évolue à J+2 voire plus.
- Les données entre la base source et le cube doivent être alignées. Or, aucune réconciliation n'est en place entre la base SQL Server et le cube OLAP.
- La base et le cube doivent être documentés (dictionnaire et schéma). Or, aucune documentation n'existe.
- Toute infrastructure de données doit être supervisée afin d'identifier les origines des anomalies. Or, ce sont les logs des tables du cube qu'on analyse.
- les données analytiques ne doivent contenir que ce qui est nécessaire à l'analyse (principe de minimisation du RGPD). Or des données personnelles sont présentes.

5.2. Causes des écarts

Les causes probables sont:

- Une absence de réconciliation entre la source et le cube OLAP : aucun contrôle ne permet de vérifier que les données du cube sont complètes par rapport à la base SQL Server.
- Un défaut de supervision technique : les traitements d'alimentation (chargement, rafraîchissement) ne sont ni tracés ni surveillés, rendant difficile l'identification des retards ou échecs.
- Un manque de gouvernance des données : l'absence de documentation et de règles de gestion (typage, contraintes) augmente le risque d'erreurs et de mauvaises interprétations.
- Un manque de maîtrise des flux et des usages des données : exposition excessive de données non nécessaires (données personnelles dans la couche BI).
- Une culture data principalement orientée vers les analyses (Data Analystes historiquement), peu d'expertise en Data Engineering, du coup, pas de supervision de la base de données.

6. Recommandations

6.1. Recommandation 1 : Mettre en place une réconciliation entre la base SQL Server et le cube OLAP

Mettre en place un processus de réconciliation entre la base source SQL Server et le cube OLAP afin de s'assurer des éventuels problèmes de synchronisation avant que les données du cube ne soient utilisées par l'équipe BI. Ce contrôle doit pouvoir comparer, pour une date de vente donnée, le nombre de ventes, le chiffre d'affaires.

Les scripts SQL fournis sont des prototypes réalisés sous PostgreSQL. Ils ne sont pas directement exécutables dans l'environnement cible SQL Server / Azure Analysis Services sans adaptation syntaxique et technique.

-- Création table simulation de la base source

```
CREATE TABLE vente_source(  
    id_vente SERIAL,  
    ean VARCHAR(200) NOT NULL,  
    date_vente VARCHAR(20) NOT NULL,  
    id_ticket VARCHAR(20) NOT NULL,  
    prix NUMERIC(12,2) NOT NULL  
);
```

```
SELECT * FROM vente_source;
```

-- ajout des clés primaires sur l'id_vente

```
ALTER TABLE vente_source  
ADD CONSTRAINT vente_source_pk PRIMARY KEY (id_vente);
```

-- création d'une vue matérialisée pour le cube OLAP qui doit être rafraîchie périodiquement

-- Cette vue sert à simuler la synchronisation entre les ventes cube et les ventes source.

```
CREATE MATERIALIZED VIEW vente_cube AS(  
SELECT * FROM vente_source  
);
```

```
SELECT * FROM vente_cube;
```

-- Création de la table de contrôle

```
CREATE TABLE controle_reconciliation(  
    id_controle SERIAL PRIMARY KEY,
```

```

    date_vente DATE NOT NULL,
    nb_vente_source INTEGER,
    nb_vente_cube INTEGER,
    nb_produit_source INTEGER,
    nb_produit_cube INTEGER,
    nb_ticket_source INTEGER,
    nb_ticket_cube INTEGER,
    ca_source NUMERIC(12,2),
    ca_cube NUMERIC(12,2),
    ecart_ca NUMERIC(12,2),
    statut VARCHAR(10) CHECK(statut IN ('CONFORME', 'ÉCART')),
    date_controle TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```
SELECT * FROM controle_reconciliation;
```

-- Procédure de réconciliation Source/Cube

```

CREATE OR REPLACE PROCEDURE reconcilier_source_cube (p_date_vente DATE)
LANGUAGE plpgsql AS $$
DECLARE
    v_nb_vente_source INTEGER;
    v_nb_vente_cube INTEGER;
    v_nb_produit_source INTEGER;
    v_nb_produit_cube INTEGER;
    v_nb_ticket_source INTEGER;
    v_nb_ticket_cube INTEGER;
    v_ca_source NUMERIC(12,2);
    v_ca_cube NUMERIC(12,2);
    v_ecart_ca NUMERIC(12,2);
    v_statut VARCHAR(20);
BEGIN
    SELECT
        COUNT(*),
        COUNT(DISTINCT ean),
        COUNT(DISTINCT id_ticket),
        SUM(ROUND(prix::numeric,2))
    INTO
        v_nb_vente_source,
        v_nb_produit_source,
        v_nb_ticket_source,
        v_ca_source
    FROM vente_source
    WHERE DATE '1899-12-30' + date_vente::INT = p_date_vente;

```

```

SELECT
    COUNT(*),
    COUNT(DISTINCT ean),
    COUNT(DISTINCT id_ticket),
    SUM(ROUND(prix::numeric,2))
INTO
    v_nb_vente_cube,
    v_nb_produit_cube,
    v_nb_ticket_cube,
    v_ca_cube
FROM vente_cube
WHERE DATE '1899-12-30' + date_vente::INT = p_date_vente;

v_ecart_ca :=v_ca_source - v_ca_cube;
v_statut := (CASE
                WHEN v_ecart_ca=0.00 THEN 'CONFORME'
                ELSE 'ÉCART'
            END);

INSERT INTO controle_reconciliation(
    date_vente,
    nb_vente_source,
    nb_vente_cube,
    nb_produit_source,
    nb_produit_cube,
    nb_ticket_source,
    nb_ticket_cube,
    ca_source,
    ca_cube,
    ecart_ca,
    statut
)
VALUES (
    p_date_vente,
    v_nb_vente_source,
    v_nb_vente_cube,
    v_nb_produit_source,
    v_nb_produit_cube,
    v_nb_ticket_source,
    v_nb_ticket_cube,
    v_ca_source,
    v_ca_cube,
    v_ecart_ca,
    v_statut
);

```

END;
\$\$;

A noter que dans le cadre de ce POC, la vue matérialisée vente_cube sert juste à simuler le principe d'un cube alimenté périodiquement depuis la source. Elle permet juste de montrer qu'un décalage peut apparaître lorsque de nouvelles ventes sont insérées après le dernier rafraîchissement.

Cette réconciliation est nécessaire car les logs sont alimentés en même temps que les tables du cube OLAP et ne permettent donc pas d'identifier un écart de synchronisation entre SQL Server et le cube OLAP.

6.2. Recommandation 2 : Mettre en place une supervision technique

En complément des contrôles métier, une supervision technique peut être mise en place afin de surveiller les performances, l'activité des connexions, l'utilisation des index et les statistiques des tables. Cette supervision vient compléter des contrôles de complétude métier et permet d'identifier en amont les anomalies techniques susceptibles d'impacter les traitements BI.

Dans l'environnement Microsoft, avec les Dynamic Management Views (DMVs) qui sont accessibles en SQL il est possible de surveiller les métadonnées, l'activité du serveur et les performances.

Voici quelques exemples de requêtes utilisées avec PostgreSQL et leurs équivalents avec SQL Server :

- **Identifier les tables utilisateur les plus modifiées :**

```
SELECT
    relname AS table_name,
    n_tup_ins AS insertions,
    n_tup_upd AS updates,
    n_tup_del AS suppressions
FROM pg_stat_user_tables
ORDER BY n_tup_ins + n_tup_upd + n_tup_del DESC;
```

L'équivalent SQL Server serait avec les DMV:

```
SELECT *
FROM sys.dm_db_index_operational_stats (NULL, NULL, NULL, NULL);
```

- **Pour identifier les requêtes en cours :**

```
SELECT *
FROM pg_stat_activity
WHERE state = 'active' ;
```

L'équivalent SQL Server serait avec les DMV:

```
SELECT *  
FROM sys.dm_exec_requests;
```

On a ainsi des informations sur les requêtes en cours d'exécution et leur état (durée, attente, session associée).

- **Pour voir uniquement les requêtes SQL bloquées :**

```
SELECT *  
FROM pg_stat_activity  
WHERE wait_event_type = 'Lock';
```

L'équivalent SQL Server serait avec les DMV:

```
SELECT *  
FROM sys.dm_exec_requests  
WHERE blocking_session_id <> 0;
```

La supervision présentée ici concerne la base de données SQL Server. Elle pourrait être étendue au cube et à la dataviz:

- Microsoft Azure Analysis Services (cube OLAP) dispose de requêtes DMVs qui utilisent une instruction SELECT et le schéma \$System avec un ensemble de lignes de schéma XML/A, par exemple `SELECT * FROM $System.<schemaRowset>`. Avec le `schemaRowset TMSchema_PARTITIONS` on peut vérifier la date du dernier rafraîchissement de chaque partition du cube, `DISCOVER_COMMANDS` donne des informations sur les commandes en cours sur le cube, `DISCOVER_SESSIONS` donne des informations sur les sessions connectées.
- Power BI expose, via son API REST et son journal d'activité, l'historique des rafraîchissements de datasets et l'utilisation des rapports.

Une supervision technique complète couvrirait donc ces trois niveaux : base de données, cube OLAP et service de dataviz.

6.3. Recommandation 3 : Restreindre les données personnelles dans la couche BI

Supprimer les colonnes non nécessaires à l'analyse BI, `hash_mdp`, `nom`, `prénom` `mail`.

On peut créer une vue dédiée à la Business Intelligence en tapant la commande SQL suivante :

```
CREATE OR REPLACE VIEW employe_bi AS(  
SELECT  
id_employe,  
employe,  
date_debut  
FROM employe  
);
```

Power BI et le cube OLAP devraient avoir accès uniquement à cette vue et non à la table complète.

6.4. Recommandation 4 : Formaliser le dictionnaire de données, documenter le modèle relationnel et les règles de calcul

Créer un dictionnaire de données officiel et documenter le schéma relationnel, les cardinalités, les clés primaires, les clés étrangères, les règles de calcul du chiffre d'affaires, les règles de rafraîchissement Power BI.

Cela permet de réduire les erreurs d'interprétation, d'améliorer la fiabilité des indicateurs, de renforcer la gouvernance des données.

6.5. Recommandation 5 : Améliorer le typage, le nommage et la qualité des données

Mettre en place des contraintes d'intégrité dans la base de données afin de garantir la qualité des données dès leur insertion :

- contraintes de type (types adaptés : NUMERIC, DATE, etc.)
- contraintes de nullité (NOT NULL)
- contraintes d'unicité (PRIMARY KEY, UNIQUE)
- contraintes de cohérence (CHECK)
- contraintes de référence (FOREIGN KEY)

Par exemple :

Champ	Règle recommandée
ean	Format texte, non nul, unique dans produit (clé primaire de la table produit)
prix	Nombre positif, format NUMERIC(12,2)
date_achat	Date valide
customer_id	Présent dans la table client (clé primaire de la table client)
id_employe	Présent dans la table employe (clé primaire de la table employe)
id_vente	Identifiant unique (clé primaire de la table vente)
id_ticket	Non nul

Voici un exemple de requête SQL :

```
ALTER TABLE produit  
ALTER COLUMN prix TYPE NUMERIC(12,2),
```

ADD CONSTRAINT prix_positif CHECK (prix > 0);

6.6. Priorisation des recommandations

Priorité	Recommandation	Urgence	Impact
1	Mettre en place une réconciliation SQL server / cube OLAP	Très élevée	Très élevé
2	Mettre en place une supervision technique	Très élevée	Très élevé
3	Restreindre les données personnelles en BI	Élevée	Élevé
4	Formaliser le dictionnaire de données et documenter le modèle relationnel	Moyenne	Élevé
5	Améliorer le typage, le nommage et la qualité des données	Moyenne	Moyen

7. Conclusion

L'audit a confirmé que le chiffre d'affaires correct du 14 août 2024 est de 284 243,88 €.

L'écart avec le premier montant observé, soit 275 186,59 €, est cohérent avec l'hypothèse d'une intégration tardive de certaines ventes ou d'un retard de synchronisation entre la source et le cube OLAP. Toutefois, en l'absence d'accès aux logs ETL, aux historiques de rafraîchissement du cube et aux journaux d'activités Power BI, la cause technique exacte ne peut pas être établie de manière certaine.

Le problème principal n'est donc pas un problème de calcul du chiffre d'affaires, mais un problème de fiabilité du processus d'intégration et de publication des données.

Les recommandations proposées visent à :

- fiabiliser les chiffres publiés dans Power BI,
- éviter l'affichage de données incomplètes,
- améliorer la détection d'anomalies,
- améliorer la gouvernance des données,
- sécuriser les données personnelles,
- documenter durablement le modèle de données.

La mise en œuvre de ces actions est importante pour restaurer la confiance dans les indicateurs BI et permettre à SuperSmartMarket de poursuivre son ambition de personnalisation client sur des bases de données fiables.

8. Annexes

Annexe 1 : Glossaire

Terme	Définition
BI	Business Intelligence, ensemble des outils et méthodes permettant d'analyser les données pour piloter l'activité
Dataviz	Data Visualization (Représentation graphique des données)
Dictionnaire de données	Document de référence décrivant l'ensemble des données d'un système d'information : nom des tables et colonnes, types, longueurs, contraintes, signification métier et règles de gestion
EAN	European Article Number (Code-barre produit)
ERP	Enterprise Resource Planning
ETL	Extract Transform Load
Hash	Hachage : résultat d'une fonction cryptographique qui transforme une donnée
Log	Journal d'activité
Modèle relationnel	Modèle de structuration des données sous forme de tables (relations) reliées entre elles par des clés primaires et étrangères.
Modèle tabulaire	Modèle analytique basé sur des tables de données, utilisé pour stocker, organiser et analyser efficacement de grands volumes de données dans Analysis Services et Power BI.
OLAP	Online Analytical Processing, technologie permettant l'analyse multidimensionnelle des données
Partition	Les partitions permettent de découper une grande table de données en plusieurs segments logiques afin de traiter (rafraîchir) uniquement les données qui changent souvent, au lieu de retraiter toute la table.
POC	Proof of Concept
Power BI	Outil Microsoft de visualisation et de reporting
Processus ETL	Consiste à extraire des données depuis différentes sources, les transformer pour les rendre cohérentes et exploitables, puis les charger dans un système décisionnel comme un entrepôt de données, un cube OLAP ou un outil BI.
RGPD	Règlement Général sur la Protection des Données
Schéma en étoile	Relier plusieurs tables de dimensions à une table de faits centrale
Table de dimensions	Table descriptive permettant d'analyser les faits selon plusieurs axes

Table de faits

Table contenant les événements mesurables, par exemple les ventes

Annexe 2 : Dictionnaires des données

Voir fichier Excel joint.

Annexe 3 : Script de création des tables du POC

-- Création des tables pour smartmarket à partir du script tiré de SQL Power Architect

```
CREATE TABLE calendrier (  
    date INTEGER NOT NULL,  
    annee INTEGER NOT NULL,  
    mois INTEGER NOT NULL,  
    jour INTEGER NOT NULL,  
    mois_nom VARCHAR(10) NOT NULL,  
    annee_mois INTEGER NOT NULL,  
    jour_semaine INTEGER NOT NULL,  
    trimestre VARCHAR(2) NOT NULL,  
    CONSTRAINT calendrier_pk PRIMARY KEY (date)  
);  
COMMENT ON COLUMN calendrier.trimestre IS 'Q1, Q2, Q3, Q4';
```

```
CREATE TABLE employe (  
    id_employe VARCHAR NOT NULL,  
    employe VARCHAR NOT NULL,  
    prenom VARCHAR NOT NULL,  
    nom VARCHAR NOT NULL,  
    date_debut INTEGER NOT NULL,  
    hash_mdp VARCHAR NOT NULL,  
    mail VARCHAR(100) NOT NULL,  
    CONSTRAINT employe_pk PRIMARY KEY (id_employe)  
);
```

```
CREATE TABLE client (  
    customer_id VARCHAR NOT NULL,  
    date_inscription DATE NOT NULL,  
    CONSTRAINT client_pk PRIMARY KEY (customer_id)  
);
```

```
CREATE TABLE produit (  
    ean VARCHAR NOT NULL,  
    categorie VARCHAR NOT NULL,  
    rayon VARCHAR NOT NULL,  
    libelle_produit VARCHAR NOT NULL,  
    prix NUMERIC(12,2) NOT NULL,  
    CONSTRAINT produit_pk PRIMARY KEY (ean)  
);
```

```
CREATE TABLE vente (  
    id_bdd VARCHAR NOT NULL,  
    customer_id VARCHAR NOT NULL,  
    id_employe VARCHAR NOT NULL,  
    ean VARCHAR NOT NULL,  
    date INTEGER NOT NULL,  
    id_ticket VARCHAR NOT NULL,  
    CONSTRAINT vente_pk PRIMARY KEY (id_bdd)  
);
```

-- Rajout des contraintes de clés étrangères sur les tables

```
ALTER TABLE vente ADD CONSTRAINT calendrier_vente_fk  
FOREIGN KEY (date)  
REFERENCES calendrier (date)  
ON DELETE NO ACTION  
ON UPDATE NO ACTION  
NOT DEFERRABLE;
```

```
ALTER TABLE vente ADD CONSTRAINT employe_vente_fk  
FOREIGN KEY (id_employe)  
REFERENCES employe (id_employe)  
ON DELETE NO ACTION  
ON UPDATE NO ACTION  
NOT DEFERRABLE;
```

```
ALTER TABLE vente ADD CONSTRAINT client_vente_fk  
FOREIGN KEY (customer_id)  
REFERENCES client (customer_id)  
ON DELETE NO ACTION  
ON UPDATE NO ACTION  
NOT DEFERRABLE;
```

```
ALTER TABLE vente ADD CONSTRAINT produit_vente_fk  
FOREIGN KEY (ean)  
REFERENCES produit (ean)  
ON DELETE NO ACTION  
ON UPDATE NO ACTION  
NOT DEFERRABLE;
```

Annexe 4 : Requête SQL du calcul du chiffre d'affaires du 14 août

-- Chiffre d'affaires du 14 août

```
SELECT
SUM(p.prix) AS chiffre_affaires
FROM vente v
JOIN produit p
    ON v.ean = p.ean;
```

/*

Le montant est très proche de 284243,88. La différence est légère.

Vérifions si ce n'est pas un problème d'arrondi.

*/

```
SELECT
    SUM(p.prix) AS ca_brute,
    SUM(ROUND(p.prix::numeric, 2)) AS ca_arrondie
FROM vente v
JOIN produit p
    ON v.ean = p.ean;
```

--On a bien le 14 août 284243,88 € de chiffre d'affaires

	ca_brute numeric	ca_arrondie numeric
1	284243.88	284243.88

Annexe 5 : Requête SQL du calcul du chiffre d'affaires par client pour le top 10 des clients

-- **chiffre d'affaires par client pour le top 10 des clients**

```
SELECT
  v.customer_id AS client,
  SUM(ROUND(p.prix::numeric, 2)) AS ca_arrondie
FROM vente v
JOIN produit p
  ON v.ean = p.ean
GROUP BY v.customer_id
ORDER BY ca_arrondie DESC
LIMIT 10;
```

	client character varying	ca_arrondie numeric
1	CUST-JNSOZSFOR...	846.86
2	CUST-GM6VBAYAB...	666.86
3	CUST-L2ST2JHI7K...	644.18
4	CUST-WU7ZKQJE4...	608.93
5	CUST-9WM83101Q...	582.03
6	CUST-ZMAOVX8XY...	576.39
7	CUST-3K66CV0OH...	571.44
8	CUST-CG23SXJDR...	531.09
9	CUST-D8IOFHVUFX...	477.35
10	CUST-IHN1HQRI7P...	463.73

Annexe 6 : Requête SQL du calcul de la part du chiffre d'affaires encaissé par employé

--la part de chiffre d'affaires encaissé par employé

-- Créons une vue pour le chiffre d'affaires par employé

```
CREATE or REPLACE VIEW ca_per_employe AS(
SELECT
v.id_employe,
SUM(ROUND(p.prix::numeric, 2)) AS ca_arrondie_par_employe
FROM vente v
JOIN produit p
      ON v.ean = p.ean
GROUP BY v.id_employe
ORDER BY ca_arrondie_par_employe DESC
);
```

```
SELECT * FROM ca_per_employe;
```

-- créons une vue pour avoir le chiffre d'affaires total sur chaque ligne qui représente un employé.

```
CREATE or REPLACE VIEW ca_arrondi AS(
SELECT
id_employe,
SUM(ca_arrondie_par_employe) OVER() AS ca_arrondi
FROM ca_per_employe
);
```

```
SELECT * FROM ca_arrondi;
```

```
SELECT
e.employe,
CONCAT(c.jour,'/',c.mois,'/',c.annee) AS date_de_début,
cap.ca_arrondie_par_employe,
CONCAT(ROUND(cap.ca_arrondie_par_employe*100.0/ca_arrondi,2),'%') AS part_ca_par_employe
FROM ca_per_employe cap
JOIN ca_arrondi ca ON cap.id_employe = ca.id_employe
JOIN employe e ON e.id_employe = ca.id_employe
JOIN calendrier c ON e.date_debut = c.date
ORDER BY part_ca_par_employe DESC
;
```

	employe character varying 🔒	date_de_début text 🔒	ca_arrondie_par_employe numeric 🔒	part_ca_par_employe text 🔒
1	aboulet	14/11/2023	7818.82	2.75%
2	ejacquier	12/6/2020	7736.16	2.72%
3	cdelisle	2/12/2020	6995.14	2.46%
4	pmanoury	29/9/2020	6616.46	2.33%
5	tarsenault	11/1/2022	6483.84	2.28%
6	adufresne	25/7/2023	6361.22	2.24%
7	adutertre	2/3/2023	6133.34	2.16%
8	pange	2/5/2021	6112.37	2.15%
9	alièvremon	12/6/2020	6111.18	2.15%
10	prodier	20/1/2021	6094.62	2.14%
11	sdeslys	17/11/2022	5989.84	2.11%
12	cgachet	21/7/2023	5936.36	2.09%
13	vbaume	3/4/2021	5909.73	2.08%
14	bcasal	17/8/2020	5827.75	2.05%
15	iescoffier	24/12/2021	5734.83	2.02%

Annexe 7 : Requête SQL pour retrouver l'écart du chiffre d'affaires

-- Vérifions les dates insertion des données de ventes qui sont indiqués dans le log

```
SELECT
l.table_insert,
CONCAT(c.jour,'/',c.mois,'/',c.annee) AS date_insertion
FROM
logs l
JOIN calendrier c
      ON l.date = c.date
WHERE l.table_insert = 'Ventes'
GROUP BY date_insertion,l.table_insert
;
```

	table_insert character varying (10)	date_insertion text
1	Ventes	14/8/2024
2	Ventes	15/8/2024

-- Vérification de l'écart de chiffre d'affaires

```
WITH ventes_log_15 AS (
SELECT
l.id_ligne AS vente,
CONCAT(c.jour,'/',c.mois,'/',c.annee) AS date_log,
v.ean
FROM logs l
JOIN vente v ON l.id_ligne = v.id_bdd
JOIN calendrier c ON l.date = c.date
WHERE l.date = '45519' AND l.table_insert = 'Ventes'
GROUP BY l.id_ligne,v.ean, date_log
)
```

```
SELECT
vl.date_log,
ROUND(SUM(p.prix::numeric),2) AS CA_update_15_aout
FROM ventes_log_15 vl
JOIN produit p
ON vl.ean = p.ean
GROUP BY vl.date_log
;
```

	date_log text	ca_update_15_aout numeric
1	15/8/2024	9057.29

Annexe 8 : Procédure de génération aléatoire de ventes

-- Procédure de génération aléatoire de ventes

```
CREATE OR REPLACE PROCEDURE prod_ventes_alea(p_nb_tickets INTEGER,p_date_vente DATE)
```

```
LANGUAGE plpgsql AS $$
```

```
DECLARE
```

```
    v_id_ticket VARCHAR(20);
```

```
    v_ean VARCHAR(200);
```

```
    v_prix NUMERIC(12,2);
```

```
    v_nb_produit INTEGER;
```

```
BEGIN
```

```
    FOR i IN 1..p_nb_tickets LOOP
```

```
        -- Générer un id_ticket unique
```

```
        v_id_ticket := 't_' || i || '_' || TO_CHAR(NOW(), 'HH24:MI');
```

```
        -- Nombre de produits aléatoire dans un ticket (1 à 10)
```

```
        v_nb_produit := FLOOR(RANDOM()*10 + 1);
```

```
        FOR j IN 1..v_nb_produit LOOP
```

```
            -- Tirer un produit aléatoirement
```

```
            SELECT
```

```
                ean,
```

```
                prix
```

```
            INTO
```

```
                v_ean,
```

```
                v_prix
```

```
            FROM produit
```

```
            ORDER BY RANDOM()
```

```
            LIMIT 1;
```

```
            -- Insérer la ligne dans la table vente
```

```
            INSERT INTO vente_source(
```

```
                ean,
```

```
                date_vente,
```

```
                id_ticket,
```

```
                prix
```

```
            )
```

```
            VALUES (
```

```
                v_ean,
```

```
                p_date_vente::DATE - DATE '1899-12-30',
```

```
                v_id_ticket,
```

```
                v_prix
```

```
            );
```

```
        END LOOP;
```

```
    END LOOP;
```

```
END;  
$$
```

Annexe 9 : Requête SQL pour la création des tables logs

-- Création table logs

```
CREATE TABLE logs(  
id_user VARCHAR NOT NULL,  
date INTEGER NOT NULL,  
action VARCHAR(10) NOT NULL,  
table_insert VARCHAR(10) NOT NULL,  
id_ligne VARCHAR NOT NULL,  
champs VARCHAR(20),  
detail VARCHAR  
);
```

-- Création table logs_client

```
CREATE TABLE IF NOT EXISTS logs_client AS (  
SELECT *  
FROM logs  
WHERE table_insert = 'Client'  
);  
ALTER TABLE logs_client  
DROP COLUMN table_insert; -- Colonne non nécessaire
```

```
ALTER TABLE logs_client  
RENAME COLUMN date TO date_insertion; --changement du nom pour éviter la confusion
```

-- Création table logs_employe

```
CREATE TABLE IF NOT EXISTS logs_employe AS (  
SELECT *  
FROM logs  
WHERE table_insert = 'Employé'  
);
```

```
ALTER TABLE logs_employe
```

```
DROP COLUMN champs,  
DROP COLUMN table_insert, -- colonne non nécessaire  
DROP COLUMN detail; -- Suppression des colonnes champs et detail qui correspondaient aux  
hash_mdp
```

```
ALTER TABLE logs_employe  
ADD COLUMN commentaire TEXT ; -- ajout d'une colonne commentaire
```

-- Création table logs_produits

```
CREATE TABLE IF NOT EXISTS logs_produits AS (  
SELECT *  
FROM logs  
WHERE table_insert = 'Produits'  
);
```

```
ALTER TABLE logs_produits  
RENAME COLUMN date TO date_insertion;
```

```
ALTER TABLE logs_produits  
DROP COLUMN table_insert; -- colonne non nécessaire
```

```
ALTER TABLE logs_produits  
ALTER COLUMN champs TYPE TEXT,  
ALTER COLUMN detail TYPE TEXT; -- changement de type nécessaire pour la fonction trigger
```

-- Création table logs_ventes

```
CREATE TABLE IF NOT EXISTS logs_ventes AS (  
SELECT *  
FROM logs  
WHERE table_insert = 'Ventes'  
);  
ALTER TABLE logs_ventes  
DROP COLUMN table_insert; -- Colonne non nécessaire
```

```
ALTER TABLE logs_ventes  
ADD COLUMN customer_id VARCHAR(200),  
ADD COLUMN id_employe VARCHAR(200),  
ADD COLUMN ean VARCHAR(50),  
ADD COLUMN date_achat::VARCHAR  
ADD COLUMN id_ticket VARCHAR(10);
```

```
CREATE TABLE logs_ventes_clean AS
SELECT
    MAX(id_user) AS id_user,
    MAX(date) AS date_insertion,
    MAX(action) AS action,
    id_ligne,
    MAX(customer_id) AS customer_id,
    MAX(id_employe) AS id_employe,
    MAX(ean) AS ean,
    MAX(date_achat) AS date_achat,
    MAX(id_ticket) AS id_ticket
FROM logs_ventes
GROUP BY id_ligne;
```

```
DROP TABLE logs_ventes;
```

```
ALTER TABLE logs_ventes_clean
RENAME TO logs_ventes;
```

Annexe 10 : Requête SQL function trigger logs_employe

-- Création de la fonction trigger pour logs_employe

```
CREATE OR REPLACE FUNCTION logs_employe_trigger_function()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN -- operation for which the trigger was fired: INSERT, UPDATE,
DELETE, or TRUNCATE
        INSERT INTO logs_employe (
            id_user,
            date,
            action,
            id_ligne,
            commentaire
        )
        VALUES (
            CURRENT_USER, -- utilisateur PostgreSQL connecté
            CURRENT_DATE - DATE '1899-12-30',
            TG_OP,
            NEW.id_employe,
            'Nouvel employé'
        );

        RETURN NEW;

    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO logs_employe (
            id_user,
            date,
            action,
            id_ligne,
            commentaire
        )
        VALUES (
            CURRENT_USER,
            CURRENT_DATE - DATE '1899-12-30',
            TG_OP,
            OLD.id_employe, -- En cas d'update, l'id ne doit pas changer
            'Modification mot de passe'
        );

        RETURN NEW;
```

```

        ELSIF TG_OP = 'DELETE' THEN
            INSERT INTO logs_employe (
                id_user,
                date,
                action,
                id_ligne,
                commentaire
            )
            VALUES (
                CURRENT_USER,
                CURRENT_DATE - DATE '1899-12-30', -- Ex pour 14/8/2024 on a 45518.
                TG_OP,
                OLD.id_employe,
                'Départ employé'
            );

            RETURN OLD;
        END IF;
        RETURN NULL;
    END;
$$ LANGUAGE plpgsql;

```

Cela sert à faire le lien avec la table calendrier qui contient toutes les dates.

-- Création du trigger

```

CREATE OR REPLACE TRIGGER logs_employe_trigger
BEFORE INSERT OR UPDATE OR DELETE ON employe
FOR EACH ROW
EXECUTE FUNCTION logs_employe_trigger_function();

```

Annexe 11: Requête SQL function trigger logs_ventes

-- Création de la fonction trigger pour les logs ventes

```
CREATE OR REPLACE FUNCTION logs_ventes_trigger_function()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO logs_ventes (
            id_user,
            date_insertion,
            action,
            id_ligne,
            customer_id,
            id_employe,
            ean,
            date_achat,
            id_ticket
        )
        VALUES(
            CURRENT_USER,
            CURRENT_DATE - DATE '1899-12-30',
            TG_OP,
            NEW.id_bdd,
            NEW.customer_id,
            NEW.id_employe,
            NEW.ean,
            NEW.date,
            NEW.id_ticket
        );
        RETURN NEW;

    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO logs_ventes(
            id_user,
            date_insertion,
            action,
            id_ligne,
            customer_id,
            id_employe,
            ean,
            date_achat,
            id_ticket
        )
        VALUES(
```

```

        CURRENT_USER,
        CURRENT_DATE - DATE '1899-12-30',
        TG_OP,
        OLD.id_bdd,
        OLD.customer_id,
        OLD.id_employe,
        OLD.ean,
        OLD.date,
        OLD.id_ticket
    );
    RETURN OLD;
END IF;
RETURN NULL;
END;
$$ LANGUAGE plpgsql;

```

-- Création du trigger

```

CREATE OR REPLACE TRIGGER logs_ventes_trigger
BEFORE INSERT OR DELETE ON vente
FOR EACH ROW
EXECUTE FUNCTION logs_ventes_trigger_function();

```

Annexe 12: Requête SQL function trigger logs_client

-- Création de la fonction trigger pour les logs clients

```
CREATE OR REPLACE FUNCTION logs_client_trigger_function()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO logs_client(
            id_user,
            date_insertion,
            action,
            id_ligne,
            champs,
            detail
        )
        VALUES (
            CURRENT_USER,
            CURRENT_DATE - DATE '1899-12-30',
            TG_OP,
            NEW.customer_id,
            'date_inscription',
            CURRENT_DATE
        );
        RETURN NEW;
    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO logs_client(
            id_user,
            date_insertion,
            action,
            id_ligne,
            champs,
            detail
        )
        VALUES (
            CURRENT_USER,
            CURRENT_DATE - DATE '1899-12-30',
            TG_OP,
            OLD.customer_id,
            'date_desinscription',
            CURRENT_DATE
        );
        RETURN OLD;
    END IF;
    RETURN NULL;
END;
```

```
$$ LANGUAGE plpgsql;
```

-- Création du trigger

```
CREATE OR REPLACE TRIGGER logs_client_trigger  
BEFORE INSERT OR DELETE ON client  
FOR EACH ROW  
EXECUTE FUNCTION logs_client_trigger_function();
```

Annexe 13: Requête SQL fonction trigger logs_produits

-- Création de la fonction trigger pour les logs produits

```
CREATE OR REPLACE FUNCTION logs_produits_trigger_function()
RETURNS TRIGGER AS $$
DECLARE
    v_champs TEXT;
    v_detail TEXT;
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO logs_produits(
            id_user,
            date_insertion,
            action,
            id_ligne,
            champs,
            detail
        )
        VALUES(
            CURRENT_USER,
            CURRENT_DATE - DATE '1899-12-30',
            TG_OP,
            NEW.ean,
            'new',
            CONCAT(NEW.libelle_produit, ',', NEW.prix)
        );
        RETURN NEW;

    ELSIF TG_OP = 'UPDATE' THEN
        v_champs:= CONCAT_WS(',',
            CASE
                WHEN OLD.libelle_produit IS DISTINCT FROM
NEW.libelle_produit
            THEN 'libelle produit'
            END,
            CASE
                WHEN OLD.prix IS DISTINCT FROM NEW.prix
            THEN 'prix'
            END
        );

        v_detail := CONCAT_WS(',',
            CASE
```

```

        WHEN OLD.libelle_produit IS DISTINCT FROM
NEW.libelle_produit
        THEN NEW.libelle_produit
    END,
CASE
    WHEN OLD.prix IS DISTINCT FROM NEW.prix
    THEN NEW.prix::TEXT
END
);
INSERT INTO logs_produits(
    id_user,
    date_insertion,
    action,
    id_ligne,
    champs,
    detail
)
VALUES(
    CURRENT_USER,
    CURRENT_DATE - DATE '1899-12-30',
    TG_OP,
    OLD.ean,
    v_champs,
    v_detail
);
RETURN OLD;
ELSIF TG_OP = 'DELETE' THEN
    INSERT INTO logs_produits(
        id_user,
        date_insertion,
        action,
        id_ligne,
        champs,
        detail
    )
    VALUES(
        CURRENT_USER,
        CURRENT_DATE - DATE '1899-12-30',
        TG_OP,
        OLD.ean,
        NULL,
        NULL
    );
    RETURN OLD;
END IF;

```

```
        RETURN NULL;
    END;
$$ LANGUAGE plpgsql;

-- Création du trigger
CREATE OR REPLACE TRIGGER logs_produits_trigger
BEFORE INSERT OR UPDATE OR DELETE ON produit
FOR EACH ROW
EXECUTE FUNCTION logs_produits_trigger_function();
```

Annexe 14 : Légende matrice évaluation des risques

MATRICE D'ÉVALUATION DES RISQUES – AUDIT BASE DE DONNÉES

Référentiel : ISO/IEC 27005 | NIST SP 800-30 | ANSSI EBIOS

LÉGENDE DES NIVEAUX DE RISQUE

CRITIQUE	Action immédiate requise (< 48 h)
ÉLEVÉ	Correction prioritaire (< 2 semaines)
MOYEN	Plan de remédiation à planifier (< 3 mois)
FAIBLE	Surveillance — Traitement lors de la prochaine maintenance

MATRICE PROBABILITÉ × IMPACT

PROBABILITÉ	IMPACT		
	Impact FAIBLE	Impact MOYEN	Impact ÉLEVÉ
Prob. ÉLEVÉE	MOYEN	ÉLEVÉ	CRITIQUE
Prob. MOYENNE	FAIBLE	MOYEN	ÉLEVÉ
Prob. FAIBLE	FAIBLE	FAIBLE	MOYEN

GUIDE DE NOTATION

PROBABILITÉ	IMPACT
Faible : exploitation difficile, pas d'outillage connu	Faible : données non sensibles, service secondaire
Moyenne : exploitation connu, accès réseau requis	Moyen : données métier exposées, dégradation partielle
Élevée : exploitation public, accès facile	Élevé : données personnelles / financières, arrêt de service

Annexe 15 : Sources

PostgreSQL create procedures :

<https://www.postgresql.org/docs/18/sql-createprocedure.html>

PostgreSQL create fonction :

<https://www.postgresql.org/docs/18/sql-createfunction.html>

PostgreSQL create trigger:

<https://www.postgresql.org/docs/18/sql-createtrigger.html>

PostgreSQL fonction trigger :

<https://www.postgresql.org/docs/current/plpgsql-trigger.html>

PostgreSQL alter table :

<https://www.postgresql.org/docs/18/sql-altertable.html>

PostgreSQL pg-stat-database-view:

<https://www.postgresql.org/docs/current/monitoring-stats.html#MONITORING-PG-STAT-DATABASE-VIEW>

PostgreSQL pg-stat-activity-view :

<https://www.postgresql.org/docs/current/monitoring-stats.html#MONITORING-PG-STAT-ACTIVITY-VIEW>

PostgreSQL pg-stat-all-indexes-view:

<https://www.postgresql.org/docs/current/monitoring-stats.html#MONITORING-PG-STAT-ALL-INDEXES-VIEW>

PostgreSQL pg-stat-all-tables-view:

<https://www.postgresql.org/docs/current/monitoring-stats.html#MONITORING-PG-STAT-ALL-TABLES-VIEW>

RGPD :

<https://www.economie.gouv.fr/entreprises/gerer-son-entreprise-au-quotidien/assurer-sa-cybersecurite-et-la-protection-de-ses/le#:~:text=dit%20la%20loi-.Le%20RGPD%2C%20qu'est%2Dce%20que%20c'est,dans%20toute%20l'Union%20europ%C3%A9enne.>

Entrepôt de données :

https://fr.wikipedia.org/wiki/Entrep%C3%B4t_de_donn%C3%A9es

Traitement analytique en ligne (OLAP) :

https://fr.wikipedia.org/wiki/Traitement_analytique_en_ligne

PostgreSQL Fonction datetime :

<https://www.postgresql.org/docs/current/functions-datetime.html>

PostgreSQL Déclaration de variable :

<https://www.postgresql.org/docs/current/plpgsql-declarations.html>

PostgreSQL Fonctions mathématiques

<https://www.postgresql.org/docs/18/functions-math.html>

Partitions dans les modèles tabulaires :

<https://learn.microsoft.com/fr-fr/analysis-services/tabular-models/partitions-ssas-tabular?view=sql-analysis-services-2025>

Superviser Azure Analysis Services :

<https://learn.microsoft.com/fr-fr/analysis-services/azure-analysis-services/monitor-analysis-services?view=sql-analysis-services-2025>

Compréhension et résolution des problèmes bloquants dans SQL Server :

<https://learn.microsoft.com/fr-fr/troubleshoot/sql/database-engine/performance/understand-resolve-blocking>

SQL Server sys.dm_exec_requests:

<https://learn.microsoft.com/fr-fr/sql/relational-databases/system-dynamic-management-views/sys-dm-exec-requests-transact-sql?view=sql-server-ver17>

SQL Server sys.dm_exec_sessions :

<https://learn.microsoft.com/fr-fr/sql/relational-databases/system-dynamic-management-views/sys-dm-exec-sessions-transact-sql?view=sql-server-ver17>

SQL Server sys.dm_db_index_operational_stats :

<https://learn.microsoft.com/fr-fr/sql/relational-databases/system-dynamic-management-views/sys-dm-db-index-operational-stats-transact-sql?view=sql-server-ver17>

DMV Analysis Services :

<https://learn.microsoft.com/fr-fr/analysis-services/instances/use-dynamic-management-views-dmvs-to-monitor-analysis-services?view=azure-analysis-services-current>

Accéder au journal d'activité de Power BI :

<https://learn.microsoft.com/fr-fr/power-bi/guidance/admin-activity-log>

API REST Power BI historique des rafraîchissements de datasets :

<https://learn.microsoft.com/fr-fr/rest/api/power-bi/datasets/get-refresh-history>